

IT-Wallet YAML Validator — Documentazione Tecnica

Script Python per la validazione automatica e la correzione dei file OpenAPI YAML degli e-service da pubblicare su PDND per il progetto IT-Wallet.

Contesto e Obiettivo

Il processo di onboarding delle Fonti Autentiche su IT-Wallet richiede che ogni ente produca un file OpenAPI YAML conforme a tre livelli di requisiti:

1. **Requisiti formali AgID/ModI** — lo YAML deve superare il checker ufficiale Spectral con le *Italian Guidelines Full* (0 errori).
2. **Requisiti strutturali IT-Wallet** — limiti di attributi (max 15 Core + 15 Dettaglio), header di sicurezza obbligatori, assenza di DPoP, presenza di Bearer Token.
3. **Requisiti semantici** — i nomi dei claim usati nell'e-service devono corrispondere esattamente al Claim Registry ufficiale della Trust Anchor IPZS.

Prima di questo script, il Service Management eseguiva questi controlli manualmente aprendo lo YAML su Swagger UI e confrontando i claim a mano. Lo script automatizza l'intero processo in pochi secondi.

Architettura dello Script

Repository GitHub: [DemPago/SM-AS](#) (privato)

Struttura del repository

Percorso	Descrizione
<code>it_wallet_validator.py</code>	Wrapper retrocompatibile — delega a <code>it_wallet_validator/__main__.py</code>
<code>it_wallet_validator/</code>	Package validator v1 (entrypoint: <code>python -m it_wallet_validator</code>)
<code>it_wallet_validator/checks.py</code>	Tutti i controlli IT-Wallet e PDND/AgID (16 controlli)
<code>it_wallet_validator/fixer.py</code>	Auto-fix struttura YAML (ruamel.yaml round-trip, preserva commenti)

<code>it_wallet_validator/report.py</code>	<code>ValidationReport</code> : raccoglie esiti, esporta HTML + JSON
<code>it_wallet_validator/claim_registry.py</code>	Fetch e parsing del Claim Registry dalla Trust Anchor IPZS
<code>it_wallet_validator/email_sender.py</code>	Invio email SMTP SSL Gmail (porta 465)
<code>it_wallet_validator_v2/</code>	Package validator v2 — aggiunge 6 controlli anti-pattern IT-Wallet
<code>it_wallet_validator_v2/checks_v2.py</code>	Controlli v2: collisione semantica, struttura credenziale, header Digest, PDND/AgID
<code>it_wallet_validator_v2/fixer_v2.py</code>	Auto-fix esteso per i nuovi controlli v2
<code>it_wallet_service_design/</code>	Package Fase 2 — genera Rulebook, schemi e payload Registry TA. Vedere documentazione dedicata .
<code>spectral-agid.yml</code>	Ruleset AgID/ModI Italian Guidelines Full v2.4
<code>package.json</code> / <code>package-lock.json</code>	Dipendenze Node.js (Spectral CLI locale in <code>node_modules/.bin/spectral</code>)
<code>requirements.txt</code>	Dipendenze Python: <code>ruamel.yaml</code> , <code>requests</code> , <code>python-jose</code> , <code>colorama</code> , <code>python-dotenv</code>
<code>.env.example</code>	Template variabili d'ambiente: <code>EMAIL_USER</code> , <code>EMAIL_APP_PASSWORD</code> , <code>MAIL_DEFAULT_RECIPIENT</code> , <code>SELF CARE_API_KEY</code>
<code>.env</code>	Credenziali reali — escluso da git

Dipendenze

Pacchetto	Uso	Tipo
<code>ruamel.yaml</code>	Parsing e serializzazione YAML	Python (pip)

	con preservazione commenti e formattazione	
<code>requests</code>	Download Claim Registry dalla Trust Anchor	Python (pip)
<code>colorama</code>	Output colorato nel terminale	Python (pip)
<code>python-jose</code>	Utility JWT (disponibile per estensioni future)	Python (pip)
<code>@stoplight/spectral- cli</code>	Validazione AgID/ModI (Italian Guidelines)	Node.js (npm locale)
<code>spectral- agid.yml</code>	Ruleset <i>Italian Guidelines Full</i> v2.4 (italia/api-oas-checker- rules) — file statico, non pacchetto	Asset di progetto

Installazione:

```

pip install -r requirements.txt
npm install    # installa Spectral da package.json
# Scarica il ruleset AgID/ModI (Italian Guidelines Full):
curl -sL https://github.com/italia/api-oas-checker-
rules/releases/latest/download/spectral.yml \
    -o spectral-agid.yml

```

i Migrazione da PyYAML a ruamel.yaml: il file `_fixed.yaml` ora preserva tutti i commenti e le quote originali del YAML sorgente. `PyYAML` non è più una dipendenza.

Struttura delle Classi e Funzioni principali

Dopo il refactoring, ogni modulo espone un sottoinsieme coeso di funzionalità.

Modulo	Componente	Responsabilità
--------	------------	----------------

<code>claim_registry</code>	<code>fetch_claim_registry(ta_url)</code>	Scarica il Claim Registry JSON dalla Trust Anchor via HTTP
<code>claim_registry</code>	<code>extract_registry_claim_names(registry)</code>	Estrae i nomi claim validi (chiavi + alias)
<code>yaml_extractors</code>	<code>_resolve_ref(ref, schemas)</code>	Risolve un <code>\$ref</code> locale verso <code>components/schemas</code>
<code>yaml_extractors</code>	<code>_collect_schema_properties(...)</code>	Raccoglie ricorsivamente tutti i nomi di property, risolvendo <code>\$ref</code> , <code>allOf</code> / <code>oneOf</code> / <code>anyOf</code> , array items
<code>yaml_extractors</code>	<code>extract_response_claims(yaml_data)</code>	Estrae i claim dai response 200/201 di tutti i path
<code>yaml_extractors</code>	<code>extract_request_headers(yaml_data)</code>	Raccoglie gli header dichiarati nei parameters, risolvendo <code>\$ref</code> verso <code>components/parameters</code>
<code>spectral</code>	<code>run_spectral_validation(yaml_content)</code>	Esegue Spectral CLI in subprocess; cerca il binario in <code>node_modules/.bin</code> / o nel PATH
<code>checks</code>	<code>check_attribute_limits(yaml_data)</code>	Verifica il limite 15 Core + 15 Dettaglio

checks	check_security_headers(yaml_data)	Controlla presenza header obbligatori; Digest solo su operazioni con request body
checks	check_no_dpop(yaml_data)	Verifica assenza di security scheme DPoP
checks	check_bearer_token(yaml_data)	Verifica presenza di scheme http/bearer
checks	check_claim_registry(yaml_data, registry)	Confronta i claim dello YAML con il registry ufficiale
checks	check_semantic_collision(yaml_data)	Rileva collisioni semantiche tra userClaims e attributeClaims
checks	check_credential_structure, check_response_integrity_headers, check_pdnd_info, check_https_servers, check_operation_ids, check_status_endpoint, check_rate_limit_headers, check_error_responses, check_dangling_refs	Ulteriori controlli PDND/IT-Wallet

<code>fixer</code>	<code>fix_yaml(...)</code>	Correzioni automatiche con ruamel.yaml round-trip (preserva commenti e quote)
<code>report</code>	<code>ValidationReport</code>	Raccoglie tutti i risultati, esporta in Markdown e JSON
<code>validator</code>	<code>ITWalletValidator</code>	Orchestratore principale: coordina le 3 fasi, gestisce input interattivo
<code>__main__</code>	<code>pick_yaml_file(se arch_dir)</code>	UI interattiva: lista YAML separati in “file yaml” e “file già corretti”
<code>__main__</code>	<code>main()</code>	Entrypoint CLI con argparse (<code>--ta-url</code> , <code>--skip-spectral</code> , <code>--dir</code> , <code>--verbose</code>)
<code>email_sender</code>	<code>send_fixed_yaml(. ..)</code>	Invio email SMTP SSL Gmail porta 465, allega <code>_fixed.yaml</code> + <code>_report.md</code>
<code>ui</code>	<code>ok</code> , <code>fail</code> , <code>skip</code> , <code>info</code> , <code>warn</code> , <code>section</code> , <code>pause</code>	Output colorato con colorama
<code>constants</code>	Costanti globali	<code>REQUIRED_SECURITY_HEADERS</code> , <code>FORBIDDEN_SECURITY_SCHEMES</code> , <code>MAX_CORE_ATTRIBUTES</code> , <code>MAX_DETAIL_ATTRIBUTES</code> ,

```
HTTP_METHODS_WITH  
OUT_BODY ,  
BODY_ONLY_HEADER  
S
```

Flusso di Esecuzione

Lo script è interattivo: ogni fase termina con una pausa (*Premi INVIO*) e un clear dello schermo.

Avvio

1. Clona il repository

```
git clone https://github.com/DemPago/SM-AS.git  
cd SM-AS
```

2. Crea e attiva il virtualenv Python

```
python3 -m venv venv  
source venv/bin/activate      # macOS/Linux  
venv\Scripts\activate        # Windows
```

3. Installa le dipendenze Python

```
pip install -r requirements.txt
```

4. Installa Spectral CLI (localmente)

```
npm install @stoplight/spectral-cli
```

Questo crea `node_modules/.bin/spectral`. Lo script lo trova automaticamente.

5. Lancia lo script

```
python3 it_wallet_validator.py
```

Fase 0 — Selezione file e Trust Anchor

- Lista tutti i file `.yaml` / `.yml` nella cartella (originali prima, poi i `_fixed` con etichetta separata)
- Esclude automaticamente i file di configurazione Spectral (es. `spectral-agid.yml`) — non sono e-service validabili
- Scarica il **Claim Registry** live da `https://pre.ta.wallet.ipzs.it/.well-known/claims-registry`

- Verifica che il documento sia **OpenAPI 3.x** — blocca immediatamente se trova la chiave `swagger` (Swagger 2.0 non supportato da PDND/ModI)

Fase 1 — Validazione AgID/ModI (Spectral)

- Esegue `spectral lint --ruleset spectral-agid.yml` con il ruleset ufficiale **Italian Guidelines Full** (v2.4, `italia/api-oas-checker-rules`)
- Se `spectral-agid.yml` non è presente nella cartella del progetto, lo script continua con le regole OAS generiche e mostra un avviso con il comando `curl` per scaricarlo
- Il ruleset usato (agid / oas-generic) è tracciato nel report JSON (`spectral.ruleset`) e nel report Markdown
- Riporta tutti gli errori (rosso) e warning (giallo) con codice e path
- Gli errori Spectral sono **non auto-fixabili** — richiedono correzione manuale
- Se Spectral CLI non è installata, la fase viene saltata con avviso

Ruleset disponibili su [italia/api-oas-checker-rules](https://italia.github.io/api-oas-checker-rules/):

- `spectral.yml` → *Italian Guidelines Full* (default consigliato, rinominato `spectral-agid.yml`)
- `spectral-security.yml` → *Extra Security Checks*
- `spectral-full.yml` → *Italian Guidelines Full + Extra Security*

Per la pubblicazione su PDND: 0 errori con *Italian Guidelines Full*.

Fase 2 — Controlli IT-Wallet

La validazione IT-Wallet è divisa in due livelli: i controlli **v1** (16 check, presenti dal package `it_wallet_validator/`) e i controlli **v2** (6 check anti-pattern aggiuntivi, nel package `it_wallet_validator_v2/`).

Controlli v1 (16)

Controllo	Regola	Auto-fix
Limiti attributi	Max 15 attributi Core + 15 Dettaglio nei response body	No
Header obbligatori (request)	Agid-JWT-Signature, Digest (solo POST/PUT/PATCH),	Sì — aggiunti come required: true

	Agid-JWT-TrackingEvidence negli header di request	
Assenza DPoP	Nessun security scheme con nome contenente dpop	Sì — scheme rimosso + riferimenti puliti
Presenza Bearer Token	Deve esistere uno scheme type: http, scheme: bearer	Sì — aggiunto BearerAuth con bearerFormat: JWT
Claim Registry	Tutti i nomi di claim nei response devono essere nel Claim Registry IPZS	No — verifica manuale
Collisione Semantica	In credenziali relazionali, claim identici non possono stare allo stesso livello in userClaims e attributeClaims	Sì — interattivo
Endpoint obbligatorio	Presenza di POST /attribute-claims/{datasetId} con datasetId, unique_id required	No
Struttura credenziale	metadataClaims required con status enum VALID/INVALID/SUSPENDED, object_id, last_updated. attributeClaims array con object_id required.	No
Header integrità response	Response 200 di POST /attribute-claims deve	No

	dichiarare Agid-JWT-Signature e Digest (INTEGRITY_REST_02 bidirezionale)	
PDND — info	<code>info.x-api-id</code> e <code>info.x-summary</code> obbligatori per pubblicazione nel catalogo PDND	No
Servers HTTPS	Tutti gli URL in <code>servers</code> devono usare https://. Blocco servers obbligatorio.	No
OperationId	Ogni operazione deve avere un operationId univoco e non vuoto. PDND lo usa per le policy di accesso.	No
Endpoint /status	Deve essere presente un endpoint di monitoring (/status, /health). PDND lo usa per verificare disponibilità AS.	No
Rate limiting headers	X-RateLimit-Limit/Remaining/Reset in ogni response non di errore. Retry-After nella response 429.	No
Error response RFC 9457	Ogni response 4xx/5xx deve usare application/problem+json con schema	No

	conforme RFC 9457 (title, status, detail).	
\$ref pendenti	Tutti i \$ref interni (#/...) devono risolvere a definizioni esistenti nel documento.	No

Controlli v2 — Anti-Pattern IT-Wallet (6 nuovi)

Disponibili lanciando `python -m it_wallet_validator_v2`. I problemi rilevati dai controlli v2 hanno **severity=warning** (non bloccanti) ma vengono auto-fixati quando possibile.

Codice	Controllo	Regola	Auto-fix
ANTI_PATTERN_NESTING_DEPTH	Max 2 livelli di annidamento	Nessuno schema nella credenziale deve superare 2 livelli di oggetti nidificati. Livello 3+ indica che si sta rispecchiando la struttura del DB.	No — warning + guida manuale
STRING_NO_MAXLENGTH	maxLength obbligatorio su stringhe	Ogni property stringa root-level deve avere $maxLength \leq 150$. Senza, i testi potrebbero non essere visualizzabili nell'App IO.	Sì — aggiunge <code>maxLength: 150</code>
STRING_MAXLENGTH_EXCEEDED	$maxLength \leq 150$	<code>maxLength</code> dichiarato ma superiore a 150.	Sì — imposta <code>maxLength: 150</code>

BLOCK_ORDER_VIOLATION	Ordine blocchi credenziale	I 4 blocchi della credenziale devono seguire l'ordine: identityClaims, userClaims, attributeClaims, metadataClaims.	Sì — riordina automaticamente
MISSING_UNIQUE_ID_REQUEST	unique_id nel requestBody	Il requestBody di POST /attribute-claims deve dichiarare unique_id come campo required.	Sì — aggiunge unique_id
MISSING_CACHE_CONTROL	Cache-Control: no-store	La response 200 delle operazioni principali deve dichiarare Cache-Control: no-store per impedire la cache dei dati sensibili.	Sì — aggiunge l'header
ANTI_PATTERN_PDF	Divieto PDF/document_url	Nessuna property deve avere nome document_url, pdf, screenshot — vietati dal modello dati IT-Wallet.	No — warning + guida manuale

Dettaglio: Controllo Collisione Semantica

Quando una credenziale contiene dati di due soggetti distinti (es. Amministratore di Sostegno e Beneficiario), avere gli stessi claim allo stesso livello in `userClaims` e `attributeClaims`

crea ambiguità. Lo script mostra i percorsi in conflitto e offre fix interattivo con oggetto nidificato. In modalità non-interattiva (stdin chiuso), il nome default usato è `beneficiary`.

Dettaglio: Struttura credenziale IT-Wallet

Verifica `metadataClaims` (lifecycle del dataset: Signal Hub e Credential Issuer dipendono da esso), `status` enum VALID/INVALID/SUSPENDED, `object_id` required in entrambi `metadataClaims` e `attributeClaims`, `attributeClaims` di tipo array.

Dettaglio: Header integrità response (INTEGRITY_REST_02)

Pattern ModI bidirezionale: la response 200 di `POST /attribute-claims/{datasetId}` deve dichiarare `Agid-JWT-Signature` e `Digest`. Senza, il Wallet non può verificare l'autenticità dei claim ricevuti.

Dettaglio: Check PDND/AgID (nuovi)

- **PDND info:** `x-api-id` identifica univocamente l'e-service nel catalogo PDND; `x-summary` è la descrizione breve obbligatoria.
- **OperationId:** PDND usa `operationId` per configurare le policy di accesso per finalità — duplicati o assenti bloccano la pubblicazione.
- **/status:** endpoint di monitoring obbligatorio per PDND e sistema IT-Wallet.
- **Rate limiting:** requisito AgID/ModI — i consumer devono sapere come gestire il throttling.
- **RFC 9457:** error response uniformi con `application/problem+json`; il 404 in particolare documenta il caso “soggetto non trovato”.
- **\$ref pendenti:** un riferimento non risolto rompe silenziosamente parser e consumer, e blocca la pubblicazione PDND.

Dettaglio: Endpoint obbligatorio POST `/attribute-claims/{datasetId}`

Fonte: Open Specification IT-Wallet, sezione 1341 — confermata da Marco Basili (PagoPA AS team).

Nota su path statici (es. file MDG AutorizzazioniGiudiceTutelare): alcuni Authentic Source usano path statici per ogni dataset (es. `/attribute-claims/INHERITANCE_ACCEPTANCE_WITH_INVENTORY_BENEFIT`) invece del path parametrico `/attribute-claims/{datasetId}`. Il validator segnala questo come errore reale (`MISSING_ATTRIBUTE_CLAIMS_PATH`) in quanto la spec IT-Wallet richiede esplicitamente il pattern con parametro dinamico. I path statici non sono conformi allo standard anche se tecnicamente validi in OpenAPI.

Il validator **non corregge automaticamente** questo errore perché la ristrutturazione dipende dall'architettura del servizio. La fix guide nel report illustra tre scenari:

Scenario	Quando usarlo	Soluzione
Caso 1 — schema uniforme	Tutti i dataset hanno la stessa struttura di risposta	Un unico path <code>/attribute-claims/{datasetId}</code> con schema generico
Caso 2a — N credenziali distinte	Ogni dataset ha una struttura <code>attributeClaims</code> diversa (es. 17 credenziali MDG)	<code>oneOf</code> con <code>discriminator.propertyName: datasetId</code> nella response 200
Caso 2b — schema aperto	Struttura non vincolabile via OpenAPI, documentata esternamente	Schema generico <code>allOf</code> con solo lo schema base, dettagli fuori spec

Fase 3 — Auto-fix e Report

I problemi auto-fixabili vengono applicati automaticamente. Vengono generati tre file di output nella stessa cartella del file di input:

File	Contenuto
<code><nome>_fixed.yaml</code>	YAML con le correzioni automatiche applicate, pronto per un secondo ciclo di validazione
<code><nome>_report.md</code>	Report leggibile in Markdown con tabelle, dettaglio problemi e suggerimenti di fix manuale
<code><nome>_report.json</code>	Report machine-readable per integrazioni CI/CD o analisi batch

Invio email del file corretto

Al termine del fix, lo script chiede se inviare il `_fixed.yaml` come allegato via Gmail:

```
Vuoi inviare il file _fixed.yaml per email? (s/n): s
Destinatario email [sm-platforms@pagopa.it]:
```

Se si preme INVIO senza digitare nulla, viene usato il destinatario default configurato nel file `.env`. È possibile sovrascriverlo digitando un indirizzo diverso.

Le credenziali Gmail vanno configurate nel file `.env` nella cartella del progetto (mai caricato su GitHub):

```
GMAIL_USER=tuamail@gmail.com
GMAIL_APP_PASSWORD=xxxx xxxx xxxx xxxx
MAIL_DEFAULT_RECIPIENT=destinatario@esempio.it
```

Nota: Gmail richiede una **App Password** (non la password normale). Si genera da myaccount.google.com/apppasswords — richiede la verifica in due passaggi attiva sull'account.

Esiti possibili

Esito	Significato	Azione
PASS	YAML conforme al 100%	Pronto per il caricamento su PDND
FIXED	Problemi auto-corretti	Verificare il file <code>_fixed.yaml</code> prima del caricamento
FAIL	Errori manuali non risolvibili automaticamente	Consultare il report per le istruzioni di correzione

Opzioni da Linea di Comando

```
python3 it_wallet_validator.py                                     #
selezione interattiva
python3 it_wallet_validator.py eservice_ente.yaml                # file
diretto
python3 it_wallet_validator.py --dir /path/agli/yaml              #
```

```

cartella specifica
python3 it_wallet_validator.py --ta-url https://ta.wallet.ipzs.it #
produzione
python3 it_wallet_validator.py --skip-spectral # salta
validazione AgID
python3 it_wallet_validator.py --verbose #
logging DEBUG

```

Trust Anchor e Claim Registry

Il Claim Registry viene scaricato in tempo reale all'avvio:

- **Pre-produzione:** <https://pre.ta.wallet.ipzs.it/.well-known/claims-registry>
- **Produzione:** <https://ta.wallet.ipzs.it/.well-known/claims-registry>

Il registry restituisce un JSON con struttura `{"claims": {"nome_claim": {...}}}`. Lo script estrae sia i nomi ufficiali che gli alias (es. `birthdate` come alias di `birth_date`). Al momento il registry conta circa **60 claim validi**. Se la Trust Anchor non è raggiungibile (timeout 10s), il controllo semantico sui claim viene saltato con avviso senza bloccare la validazione.

Bug Corretti Rispetto al Prototipo Iniziale

#	Bug	Impatto sul risultato
1	Precedenza operatori errata in <code>check_bearer_token:</code> or v.get("bearerFormat") is not None fuori dall'and	Falso OK — qualsiasi scheme con <code>bearerFormat</code> (es. OAuth2) faceva passare il check
2	<code>_collect_schema_properties</code> non risolveva <code>\$ref</code> annidati oltre il primo livello	Falso OK su UNKNOWN_CLAIMS — claim dentro schemi referenziati venivano ignorati
3	<code>extract_request_header</code> ignorava header	Falso FAIL su MISSING_SECURITY_

	definiti in components/parameters e referenziati con \$ref	HEADERS — header presenti ma non trovati
4	fix_yaml aggiungeva Digest anche su GET, HEAD, DELETE che non hanno request body	YAML corrotto — header semanticamente errato su operazioni senza body
5	Nessuna verifica della versione OpenAPI	Swagger 2.0 veniva processato producendo risultati casuali (struttura incompatibile)
6	check_security_headers non risolveva \$ref nei parametri per il controllo Digest	Falso FAIL + header Digest duplicato nel _fixed.yaml su operazioni che lo dichiaravano già via \$ref
7	check_semantic_collision si fermava al primo path con response 200 (es. /status con schema Problem)	Collisione semantica mai rilevata — lo schema CredentialClaimsResponse non veniva mai raggiunto
8	send_fixed_yaml calcolava il path del report come <code><stem>_fixed_report.md</code> invece di <code><stem>_report.md</code>	Il _report.md non veniva mai allegato all'email nonostante il codice sembrasse farlo
9	check_required_endpoint: \$ref non risolto nel	Falso positivo su YAML con \$ref valido ma non

	request body produceva schema <code>{}</code> e segnalava sempre MISSING_UNIQUE_ID	presente nei components locali
10	ValidationError: <code>colliding_claims</code> e <code>items_schema_ref</code> aggiunti dinamicamente fuori dal costruttore; <code>items_schema_ref</code> mai letto	Attributi non dichiarati nel tipo, codice morto e potenziale AttributeError su iterazione
11	fix_yaml: \$ref items non risolto causava <code>continue</code> silenzioso senza avviso	Incapsulamento claim saltato senza nessun feedback all'utente
12	check_security_header s: Digest marcato OK se anche solo 1 operazione con body lo dichiarava, ignorando le altre	Falso OK — operazioni POST/PUT/PATCH senza Digest non venivano segnalate
13	<code>_resolve_p</code> definita come nested function identica in 2 punti diversi del codice	Duplicazione e rischio di drift nella logica di risoluzione \$ref
14	Nessun check <code>isinstance(yaml_d ata, dict)</code> dopo yaml.safe_load; import <code>base64</code> inutilizzato; dead code <code>else/pass</code> in	Crash silenzioso su YAML non-dict; codice confuso

	check_semantic_collision	
15	<code>pause()</code> e tutti gli <code>input()</code> in <code>validator_v2.py</code> non gestivano <code>E0FError</code>	Crash con traceback quando lo script veniva lanciato in modalità non-interattiva (es. pipeline, test automatici, stdin chiuso)
16	Report Markdown: i warning auto-fixabili (<code>STRING_NO_MAXLENGTH</code>) apparivano sia in “ Corretti automaticamente” che in “ Warning”	Duplicazione nel report: un problema auto-fixato veniva presentato come warning non risolto
17	<code>check_nesting_depth</code> : ricorsione infinita su schemi con strutture cicliche o <code>\$ref</code> autoreferenziali	Stack overflow su YAML con schemi profondi o mal formati
18	<code>_schema_max_depth</code> : i rami dell'albero di annidamento non erano indipendenti — profondità' calcolata sommando livelli di branch diversi	Falso positivo su schemi con più property parallele di profondità' 1 — venivano erroneamente segnalati come troppo profondi

Limitazioni Attuali e Sviluppi Futuri

Limitazione	Note
Nessuna modalità batch	Validare più YAML in sequenza con report consolidato

Nessun controllo su <code>info.version</code>	PDND richiede semantic versioning x.y.z
Ruleset Spectral non aggiornato automaticamente	<code>spectral-agid.yml</code> è un file statico versione 2.4 — aggiornare manualmente a ogni nuova release di <code>italia/api-oas-checker-rules</code>
Fase 2 — Service Design	Implementata nel package separato <code>it_wallet_service_design</code> . Genera automaticamente Rulebook EUDI (8 capitoli), JSON Schema, CDDL e 5 payload Registry TA a partire dallo stesso YAML e-service. Vedere IT-Wallet Service Design — Documentazione Tecnica .

Riferimenti

- [pagopa/PLSM-AS](#) — Repository GitHub del progetto
- [italia/api-oas-checker-eservice](#) — microservizio FastAPI + Spectral, fonte del validator wrapper
- [italia/api-oas-checker-rules](#) — ruleset Spectral ufficiali AgID (Italian Guidelines)
- [Italian OpenAPI Checker](#) — versione web del checker per verifica manuale
- [Trust Anchor pre-prod IPZS — endpoint discovery](#)
- Linee Guida Modello di Interoperabilità Tecnica PA — AgID